Reading the Architecture off the Weights: Manifest-Only Recovery of Neural Network Anatomy

Sattaya Singkul
Researcher
joeysattaya@gmail.com

Abstract

Engineers who deploy a model need to know what it is made of: which blocks repeat, what each block computes, how the parts connect and where the parameters sit. This information is usually spread over model cards, family-specific configuration files and modeling code. We show that most of it can be read from the tensor manifest alone, the list of tensor names, shapes and storage types that a safetensors file keeps in a header of a few tens of kilobytes. Our method normalizes quantized storage, finds repeated blocks in the module tree, assigns roles to the parts of each block from the algebra of their shapes, and links components by matching their widths. Fields that shapes cannot determine, such as the number of attention heads, are read from `config.json` when it exists, and the configuration is checked against the weights. On 510 architectures from the `transformers` library, with ground truth taken from module classes that the method never sees, per-tensor role accuracy is 96.8%, against 69.3% for name-pattern rules, and 95.0% when every name is replaced by a random token. Parameter counts are exact for every model, including under NVFP4, FP8 and GPTQ storage. The method runs in about 11 ms per model in a browser, and we release it as an open explorer that reads any Hugging Face repository without downloading weights.

1 Introduction

Anyone handed a new checkpoint asks the same questions. How many layers does it have, and are they all alike? Is it a mixture of experts, and how much of its size is experts? Which part runs once per input and which part runs once per generated token? For a well-known family, `config.json` answers these questions. For anything new it often does not. Keys differ between families (`hidden_size`, `d_model`, `n_embd`), composite models nest several schemas, and fields such as `layers_block_type` only make sense after reading the modeling code. Reading that code is slow, and running it through `trust_remote_code` is a known risk on public hubs [31]. Graph viewers such as Netron [22] or Model Explorer [7] need an exported graph and, in practice, the whole file.

A much smaller description is almost always available. A safetensors file [13] starts with an 8-byte length followed by a JSON header that lists the name, dtype, shape and byte range of every tensor. Two HTTP range requests are enough to fetch it. Across the 510 architectures studied here its median size is 46 KB, about 6×10^{-5} of the weights in BF16. PyTorch zip checkpoints keep the same information in their `data.pkl` record, which can be read without touching tensor data.

This paper asks how much of a model's architecture can be recovered from the manifest alone, and how reliably. The question is practical: a tool that answers it can explain a hub model in a browser tab before anyone downloads it. It also has an analytical side, because some properties leave no trace in tensor shapes. An honest tool should say which ones, measure how often a default guess for them is right, and take them from a better source when one exists.

Our starting point is that a manifest carries two kinds of evidence. The name tree shows structure: integer-indexed lists are stacks of blocks, identical subtrees are identical block types, and the module that owns a stack marks a component. The shapes show function. All blocks in a stack share a residual width d. An attention region is a set of projections that leave d and one that returns to it. A mixture of experts is a set of tensors whose leading dimension equals the output size of a router. A selective state-space mixer is a depthwise convolution next to a few per-head vectors. Two components are connected where the output width of one equals the input width of the other. Names help to break ties, but, as Section 8 shows, most of the work survives when every name is replaced by a random token.

The paper makes three contributions. First, we define manifest-only architecture recovery, give a representation of a model's anatomy that is useful to engineers, and analyse which of its properties shapes cannot identify (Section 3). Second, we describe a recovery algorithm that combines module-tree structure with shape algebra and width matching, handles six quantized storage layouts, and fuses non-identifiable fields from the configuration while checking it against the weights (Sections 4 and 5). Third, we evaluate it on 510 architectures with independent ground truth and release it as a bilingual web tool (Sections 6–9).

2 Related work

A checkpoint usually comes with three descriptions of the same model: the weights, a configuration file and code. Model cards [16] add a human summary whose architectural fields are free text. The configuration is authoritative for the library that wrote it, but its schema depends on the family and, for new families, has to be interpreted through code. Pickle-based weight formats can execute code when loaded [31]. Safetensors avoids this, and its header is the most uniform self-description that exists across families. Our work treats that header as the primary source.

Netron [22], TensorBoard and Model Explorer [7] draw operator graphs from exported formats. They show a graph faithfully but do not group operators into semantic parts, and they need the full file. Transformer Explainer [2] explains one fixed architecture interactively. Our tool borrows its step-by-step style but derives the content from the manifest, so it applies to any architecture.

Architecture recovery has been studied mostly as an attack. Side-channel methods reconstruct a network from cache timing or memory traces of a running model [12, 28]; in that setting the observer has no access to the file. Here the header is public, and the difficulty lies in interpreting it. Weight fingerprinting such as HuRef [30] identifies a model's lineage from weight values, which we never read.

The shape rules encode the signatures of attention and its variants, including grouped-query [1] and latent attention [4], sparse mixtures of experts [5, 24], selective state-space and linear-attention mixers [3, 9, 29] and their hybrids [17], conformers [10, 21], transducers [8, 27], diffusion transformers [19] and tabular in-context learners [11, 20]. The storage rules cover GPTQ [6], AWQ [14], FP8 [15], MXFP4 [23], NVFP4 [18] and the MLX and bitsandbytes layouts.

3 Problem formulation

A manifest is a set $\mathscr{M} = \{(n_i, s_i, \tau_i)\}$ of tensor names (dot-separated paths), shapes and storage dtypes. A configuration $\mathscr{C}$, a nested key–value tree, may also be available. The goal is an anatomy $\mathscr{A}$ with four levels. At the top are components, such as a vision tower, a projector, a language backbone or a prediction network, each with a residual width d, a modality and a role, and connected by directed edges that form the data path. Each component contains stacks of repeated blocks, and each block is assigned a block type. Each block type is divided into regions, labelled with one of nine roles: attention, cross-attention, feed-forward, mixture of experts, SSM or linear attention, convolution module, adaLN modulation, recurrent cell and norm. Tensors outside blocks are labelled input stem, output head or adapter. Finally, the anatomy carries logical shapes

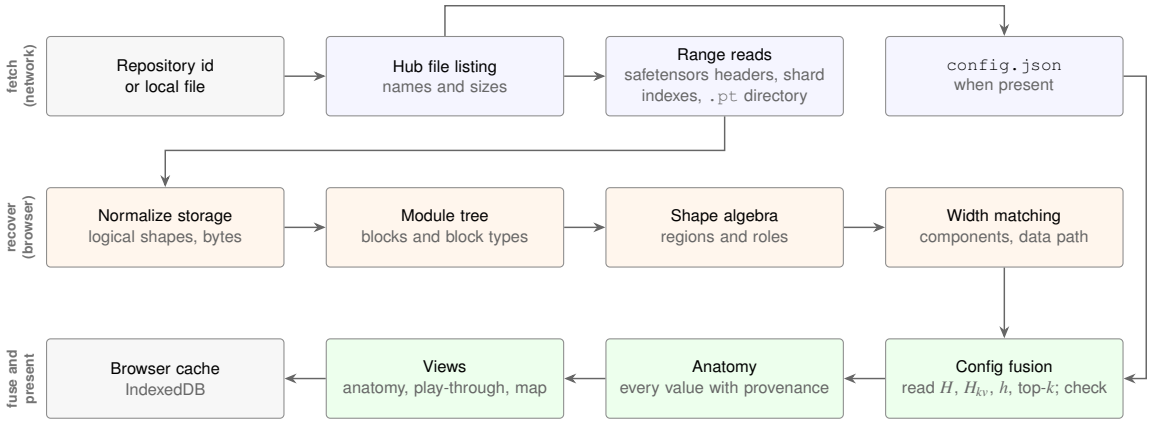


Figure 1: Workflow. Only descriptive bytes are fetched: the file listing, safetensors headers, shard indexes, the zip directory and pickle record of PyTorch checkpoints, and the configuration. Recovery and fusion run in the browser, and results are cached locally.

under quantized storage and derived quantities: parameters, stored bytes, KV-cache bytes per token and active parameters per token. Every architectural field records where it came from: measured from the manifest, read from $\mathscr{C}$ and checked against the manifest, or estimated.

3.1 What a manifest cannot identify

Some properties leave no trace in shapes. The most important is the split of attention width into heads. A projection $W_q \in \mathbb{R}^{Hh \times d}$ with H heads of size h has the same shape as one with $H/2$ heads of size $2h$, so only the product Hh and the ratio of query to key–value width are visible. The pair becomes identifiable only through per-head parameters, such as query and key norms of width h or relative-position biases of shape $H \times h$. Masking is also invisible: a decoder-only language model with tied embeddings stores exactly the tensors of a bidirectional encoder of the same size. The number of experts is visible but the number used per token is not. Gated and ungated feed-forwards differ in the number of matrices, but SiLU and GELU do not differ at all, and pre-norm and post-norm blocks store identical tensors.

These limits shape the design. The algorithm reports such fields only as estimates, the interface labels them as estimates, and when a configuration exists the fields are read from it and checked against the shapes (Section 5.5). The evaluation measures both how often the estimate is right and how much the configuration adds.

4 System overview

Figure 1 shows the workflow. Given a repository id, the explorer lists the repository and decides which files describe the weights. Safetensors files are preferred. Their shard indexes are read per folder, so a diffusers repository with `transformer/`, `text_encoder/` and `vae/` folders yields one name prefix per component. For each safetensors file, two range requests fetch the length and the header; up to eight files are read in parallel. If a repository only has PyTorch zip checkpoints, the reader fetches the last 64 KB of the file to find the zip central directory, then fetches only the `data.pkl` record and interprets it with a restricted unpickler that rebuilds names, shapes and dtypes without resolving any Python callable. The configuration is fetched alongside. The manifest then passes through the recovery algorithm of Section 5 and the result through configuration fusion. Each step is shown to the user as it happens (“reading header 3 of 12, 41 KB so far”), and the final anatomy is cached in the browser.

Algorithm 1 Manifest-only anatomy recovery with configuration fusion

Require: manifest $\mathscr{M}$, optional configuration $\mathscr{C}$
$\mathscr{T} \leftarrow$ NORMALIZE($\mathscr{M}$) ▷ logical shapes, stored bytes
build the module tree; find lists, expert lists, sub-layer lists, block lists and their owners
for all block list L **do**
    group items by structural signature into block types
    for all block type β **do**
        $d \leftarrow$ width of the leaf norms of a representative block
        classify its children by the rules of Table 1; split nested blocks; pair halves
    end for
end for
label stems and heads; find adapters, transducer joints, timestep embedders, message passing
choose the backbone and the family; derive d, L, f, E, V and KV bytes per token
if $\mathscr{C}$ is given **then**
    match sub-configurations to components by width and depth; read H, H_{kv}, h, top-k
    check $Hh = x$ and $H_{kv}h = x_{kv}$; mark each field as read from $\mathscr{C}$ or report the mismatch
end if
return the anatomy, with provenance and evidence for every decision

5 Recovering the anatomy

The rules in this section read only shapes, dtypes and the structure of the name tree. A small lexicon of name fragments (`attn`, `mlp`, `experts` and similar) breaks ties that shapes leave open. It can be switched off, and Section 8 reports results with and without it. Algorithm 1 summarizes the procedure.

5.1 Normalizing storage

Quantized checkpoints store one logical weight as several tensors. We group tensors by module and recognize the layout from dtypes and shape ratios before looking at names. NVFP4 stores a `U8` tensor of shape $[o, i/2]$, an FP8 scale tensor $[o, i/16]$ and a scalar second-level scale, so the logical shape is $[o, i]$ and the group size follows from the ratio. FP8 stores an FP8 weight with a scalar, per-channel $[o, 1]$ or block scale. GPTQ stores an `I32` tensor $[i/8, o]$ with scales $[i/g, o]$; AWQ packs the other axis, and the two are told apart by the axis the scales share. MXFP4, MLX and bitsandbytes layouts follow the same pattern. Auxiliary tensors count toward stored bytes but not toward parameters.

5.2 Blocks from the module tree

Names form a tree. A node whose children are all integers, or keys such as `expert_0`, `expert_1`, is a list. An item is composite if its subtree holds at least two modules with parameters; this separates blocks from `nn.Sequential` wrappers around single layers. Expert lists are lists inside an item of another list whose items are identical two- or three-matrix MLPs without norms, usually beside a router whose first dimension equals the number of items. Stacked experts appear instead as 3-D tensors $[E, \cdot, \cdot]$ with one trailing dimension equal to the block width. Block lists are the innermost composite lists that are not expert lists, with one correction for sub-layers: when every item of an outer list holds short inner lists of equal length, as in T5's `block.`i`.layer.`j or LongCat's two attention and two MLP sub-layers per layer, the inner items are sub-layers and the outer items are the blocks.

Each block list belongs to the module that owns it, raised through its ancestors while no other stack lives there and, in multi-folder repositories, never above the folder. These owners become components, so that embeddings and heads next to an `encoder` wrapper join the right component. Within a list, each block

Role	Shape signature inside a block of residual width d
Mixture of experts	stacked experts $[E,\cdot,\cdot]$ or an expert list, with a router $[E',d]$, $E \le E' \le E+8$
SSM / linear attn.	depthwise conv $[C,1,k]$, $k \le 8$, with recurrence parameters on the mixer: two or more vectors of one length $n_h \neq d$, or a state tensor with a small last dimension; without a conv, two or more per-head gates $[\le 64,d]$ or decay vectors
Feed-forward	one output $[d,f]$ and one or two inputs $[f,d]$, or a fused input $[2f,d]$ with $f > 1.3d$
Attention	one output $[d,x]$ and inputs from d, or a fused input $[x+2x_{kv},d]$; low-rank chains that stay inside the region count as internal
Cross-attention	as attention, with at least two projections from another component's width $w \ge 128$
Recurrent cell	two or more matrices with $3h$ or $4h$ rows (GRU, LSTM), none returning to d
Convolution module	depthwise conv with pointwise $[C,C',1]$ convs or batch-norm statistics
adaLN modulation	one matrix $[kd,t]$ from a conditioning width $32 \le t < d$, or $k \in \{6,9,12,18\}$ when $t = d$, or a $[1,k,d]$ table

Table 1: Region signatures. A matrix $[o,i]$ is an input to the region if $i = d$ and an output if $o = d$.

gets a signature, the sorted multiset of (relative path with indices masked, tensor rank) pairs, and identical signatures form a block type. In a hybrid stack such as Nemotron-H [17] this gives one type each for the Mamba-2, MoE and attention layers, and the sequence of types is the layer pattern.

5.3 Regions by shape algebra

For each block type we analyse one representative block. Its residual width d is the length of the vectors held by its leaf norm modules. This is more robust than counting matrix dimensions, which the inner width of an SSM can dominate. The children of the block are then classified with the signatures in Table 1. For attention, vectors in norm-only submodules whose length divides x give the head size, and parameters of shape $[H,h]$ with $Hh = x$ give both factors.

Four refinements matter in practice. A child that holds two sub-modules of different confident roles, such as an attention and an MLP inside one "inducing block", is split. Two half feed-forwards in sibling modules ($[f,d]$ in `intermediate` and $[d,f]$ in `output`) are paired. If a region is weak in the usual layout but becomes attention when every matrix is transposed, the weights are stored as $[\text{in},\text{out}]$, as GPT-2's `Conv1D` layers are. Finally, a list whose items map d to another width and never return to d is not a residual stack: its items are read as per-layer prediction heads, such as the box MLPs of DETR-style detectors, or as projectors when the output width belongs to another component, as with the DeepStack mergers of Qwen3-VL.

5.4 Stems, heads and the data path

Outside blocks, a matrix $[V,d]$ with $V > d$ is a token table and its twin an untied output head. A vector of length V is the head's bias, and a small table beside a token table or a convolutional stem is a position or type embedding. A chain of matrices whose free input width equals the width d_X of one component, or $k^2 d_X$ for $k \in \{2,3,4\}$ after pixel shuffling, and whose free output width equals the width d_Y of another, becomes an adapter with edges $X \to \text{adapter} \to Y$.

Three further patterns become components of their own. A transducer has a label embedding $[V,h_p]$ next to a recurrent cell, an encoder projection $[j,d_{\text{enc}}]$ and an output $[V',j]$ with $V \le V' \le V+16$; the excess $V'-V$ is the number of duration bins of a token-and-duration transducer [27]. A diffusion denoiser has a timestep MLP from a sinusoidal width (128, 256 or 320) to a width t that feeds the modulation matrices in its blocks. A group of square maps $[C,C]$ with an aggregation matrix $[C,rC]$ is a message-passing layer with r reducers. The backbone is the largest component that owns a token table or output head, and the family

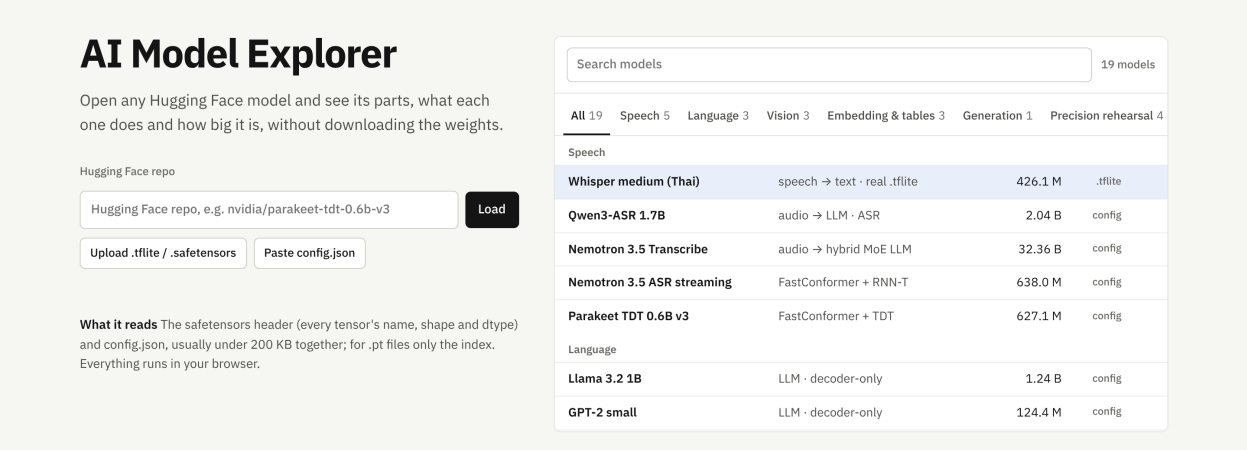


Figure 2: Entry screen. A repository id, an uploaded `.safetensors` or `.tflite` file, or a pasted configuration starts an analysis. The library on the right holds analysed models grouped by task.

(decoder, prefix LM, encoder–decoder, transducer, diffusion denoiser, encoder) follows from the components and edges.

5.5 Configuration fusion and consistency checking

When a configuration is available, the fields of Section 3.1 are read from it. Configurations of composite models nest several sub-configurations, so we first decide which one describes which component. Every object in $\mathscr{C}$ that declares a width (`hidden_size`, `d_model`, `n_embd`, `dim` and similar) is a candidate. A component is matched to the candidate whose width equals its residual width, preferring one whose layer count equals the component's and whose path names the right kind of component (`text`, `llm`, `vision`, `audio`). From the match we read the head count, the key–value head count, the head size and the routing top-k.

The match is then checked against the weights: the head count times the head size must equal the width x of the attention projections, and the key–value product must equal x_{kv}. A field that passes is shown as read from the configuration. A mismatch is reported rather than resolved, because it usually means the field has a different meaning in that family (Section 8.4). Without a configuration, the head size falls back to 128 for $d \geq 2048$ and 64 otherwise, and is shown as an estimate.

Parameters and stored bytes are sums over logical and stored tensors. KV-cache bytes per token are $2b\sum_{\ell \in \text{attn}} x_{kv}^{(\ell)}$ for b bytes per element; SSM layers keep a constant state instead. With a top-k value, active parameters per token are the non-expert parameters plus k/E of the experts. The whole procedure makes a few passes over the tree and runs in time close to linear in the number of tensors.

6 The explorer

The algorithm is the core of an open web tool [25]. We first built it for a workshop on edge inference, where participants had to understand a speech model before compressing it, and later generalized it to any architecture. Three goals guided the interface. The first screen should answer what the model is made of and how large each part is. Each value should say where it comes from, so that a reader can tell a measurement from a configuration value or an estimate. And a newcomer should be able to walk through the model from input to output, seeing what each part receives and produces.

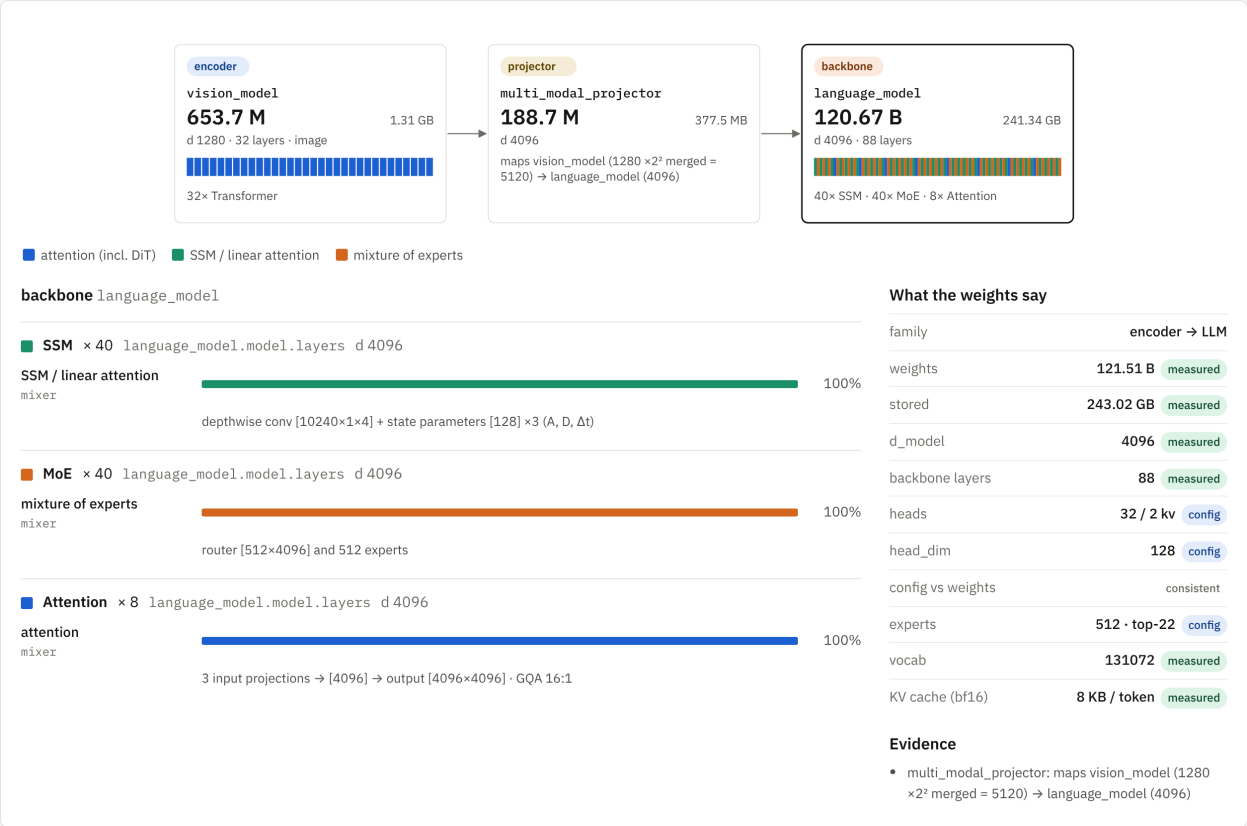


Figure 3: Anatomy view of Nemotron 3.5 Super VL (121.5 B). The three components appear in data-path order, and the backbone's strip shows how SSM, MoE and attention layers interleave. Below are the block types with the share of parameters per region and the evidence for each label; on the right, the values read from the model with their provenance and the result of the configuration check.

The entry screen (Figure 2) offers three ways in and a library of analysed models. The anatomy view (Figure 3) draws the components in data-path order. Each box gives the component's role, parameters, stored size, width and depth, and a strip with one cell per layer coloured by block type, so a hybrid backbone can be read at a glance. Selecting a component lists its block types with the share of parameters per region and the shape evidence behind each label. A side panel lists the values read from the model, each tagged as measured, read from the configuration or estimated, and states whether the configuration agrees with the weights.

The big-picture view (Figure 4) lays the model out in columns from input to output. Pressing Play steps through the columns. For each one, a panel shows what goes in and what comes out, with small stand-ins for the data (a waveform, a spectrogram, a row of tokens, a matrix), the tensor shapes, and one sentence on why the part matters for optimization. The loop at the end follows the family: next token for decoders, next frame for transducers, next denoising step for diffusion models. A layer map draws every layer at its true size under switchable lenses (size, bytes read per sentence, precision), and a report lists where parameters, compute and memory traffic go, together with short observations such as "experts hold 92.8% of the weights; about 13.6 B are active per token". The interface is available in Thai and English.

As an example of use, suppose an engineer wants to run `nvidia/parakeet-tdt-0.6b-v3` on a phone. They paste the repository id and press Load. The explorer lists the repository, fetches `config.json` and the 63 KB header of the 2.5 GB weight file, and finishes in well under a second on an ordinary connection. The anatomy view shows three components: a 24-block conformer encoder with 97% of the parameters, a 12 M prediction network with a two-layer LSTM, and a 6 M joint network whose output size, 8198, is the

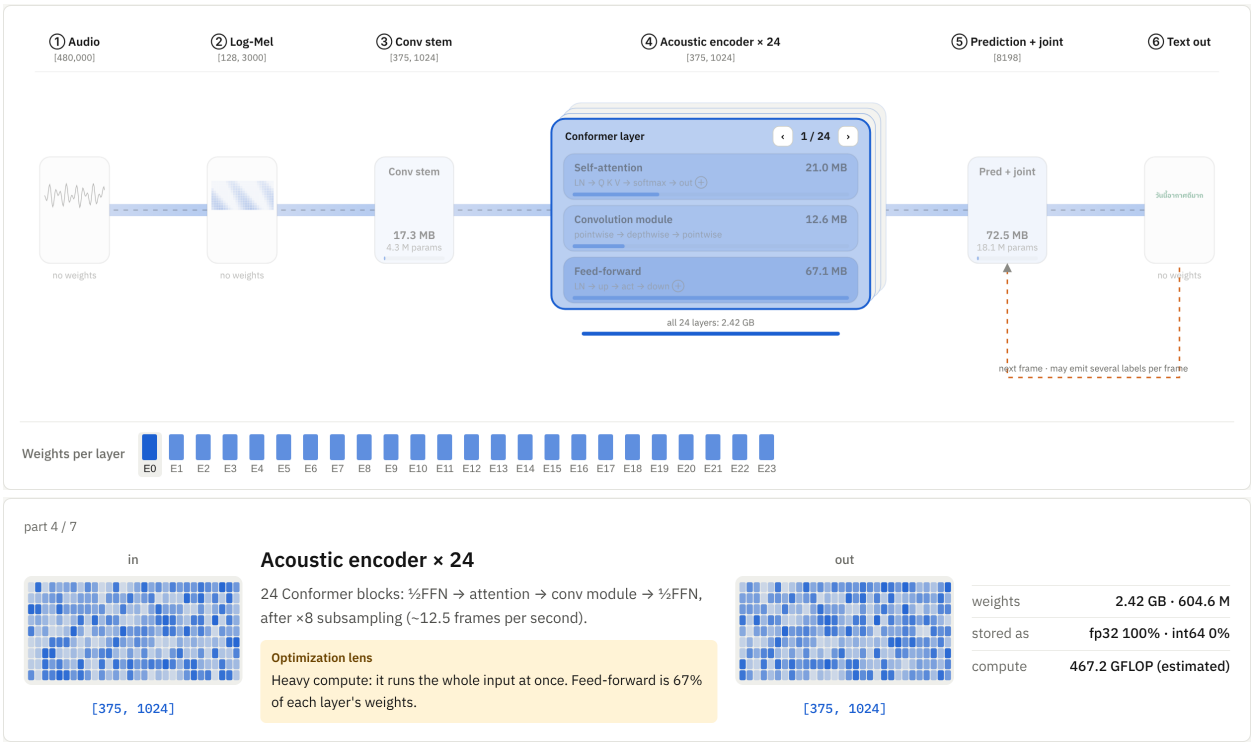


Figure 4: Big picture and play-through for Parakeet TDT 0.6B v3. Top: the transducer from audio to text, with the per-frame loop drawn back to the prediction and joint networks. Bottom: step 4 of 7, the acoustic encoder, with its input and output shapes, size, precision and estimated compute.

label vocabulary plus five duration bins. The block view shows that feed-forward modules hold two thirds of each conformer block, which is where pruning or low-bit storage pays off. The play-through then makes clear that the encoder runs once per utterance while the prediction and joint networks run once per emitted label, so streaming latency depends on the small networks rather than on the encoder.

The tool is a single static page with no server, about 3,000 lines of JavaScript. The recovery code is a separate module shared with the evaluation harness, so the numbers in Section 8 measure exactly the code that users run.

7 Evaluation setup

We ask five questions. How accurately are per-tensor roles recovered, and how much depends on names (RQ1)? Are stacks, block counts and block types recovered (RQ2)? Does quantized storage change the results (RQ3)? Which hyperparameters can shapes identify, how much does the configuration add, and do configurations agree with their weights (RQ4)? What does recovery cost (RQ5)?

Corpus. The `transformers` library, version 5.22.1 [26], registers 788 configuration types. For each, we instantiate the default configuration with the most specific model class available, on PyTorch's meta device, which creates shapes without allocating memory. 548 types instantiate, and removing identical manifests leaves 510. Tied parameters are counted once and persistent buffers are kept, as in a saved checkpoint. The corpus has 268,040 tensors, a median of 419 per model, and spans 1.5 K to 1.74 T parameters with a median of 473 M. Of the 510 models, 475 contain attention, 73 mixtures of experts, 27 SSM or linear-attention

	Role accuracy (%)		Block lists (%)				Params
Condition	macro	micro	recall	precision	count	types	exact
Names and shapes	**96.8**	**99.9**	98.6	91.4	100	92.5	510/510
Shapes only	95.0	99.8	98.6	91.4	100	92.5	510/510
Name-pattern rules	69.3	8.4	–	–	–	–	–
Name-pattern rules, names hashed	0.0	0.0	–	–	–	–	–
NVFP4 storage	96.8	99.9	98.6	91.4	100	92.5	510/510
FP8 storage	96.8	99.9	98.6	91.4	100	92.5	510/510
GPTQ storage	96.8	99.9	98.6	91.4	100	92.5	510/510
Shapes only, NVFP4 storage	95.0	99.8	98.6	91.4	100	92.5	510/510

Table 2: Main results on 510 architectures. *Count* is the share of matched lists with the exact number of blocks and *types* the share whose block-type partition equals the ground truth. The name-pattern rules produce no stacks. Their micro accuracy is low because the three largest models, all trillion-parameter mixtures of experts, dominate the pooled score and their experts are mislabelled.

mixers, 140 convolutional parts and 70 cross-attention; 231 have more than one stack and 153 have a stack with more than one block type. The configuration of each model is kept for RQ4.

Ground truth. Labels come from information the algorithm never sees: the Python class of each module. A tensor's role is given by its nearest ancestor whose class name ends in a recognized suffix (`Attention`, `MLP`, `SparseMoeBlock`, `Mamba2Mixer`, `RMSNorm` and so on), with MoE containers overriding their children and a short list of module-name fallbacks for loose tensors. Tensors that match nothing are labelled *other* and excluded. Convolutional stems and position tables outside block stacks are merged into one input-stem role, because the library is inconsistent about which class owns them. Block lists are module lists whose children each hold at least two modules with parameters, excluding expert containers and using the same sub-layer convention as the algorithm. Block types are the distinct class compositions of the items. Hyperparameters come from the configuration, using the text sub-configuration for multimodal models.

Metrics and conditions. Per-tensor role accuracy is weighted by parameter count within each model and then averaged over models (macro, so that each architecture counts once) or pooled over all tensors (micro, which the largest models dominate). We also report per-role F1 over tensors, recall and precision of block lists by path, the share of matched lists with the exact block count and the exact type partition, exact-match accuracy of hyperparameters, and exactness of the parameter total. The full method uses names and shapes. The shapes-only condition replaces every non-numeric path token by a consistent random hash and switches the lexicon off, which removes all lexical evidence but keeps the tree; it approximates checkpoints exported by unfamiliar code. The quantized conditions rewrite every 2-D linear weight as NVFP4, FP8 with per-channel scales, or GPTQ 4-bit (Appendix B). The baseline is a careful set of name-pattern rules of the kind most tools use, taken from the previous version of our own explorer: regular expressions for layer indices, `q_proj`, `query`, `c_attn`, `mlp`, `fc1`, `encoder_attn`, `mixer` and so on.

8 Results

8.1 RQ1: Roles

Per-tensor role accuracy is 96.8% macro (Table 2). The distribution over models is skewed: the median model is labelled perfectly, 414 of 510 models reach 99% and 473 reach 90%, while 10 fall below 50%.

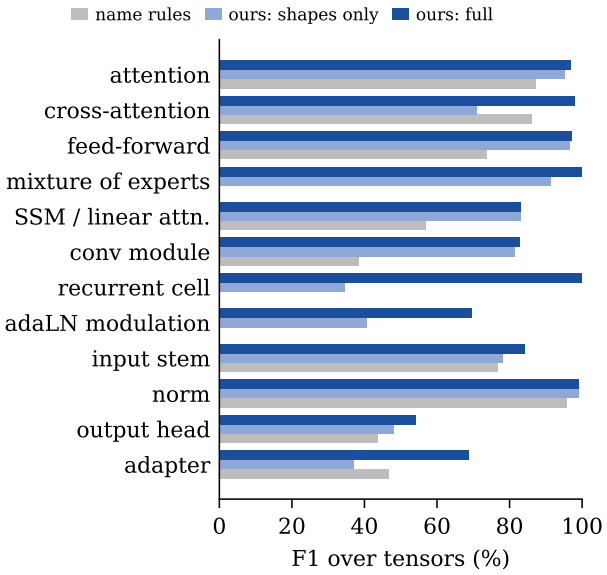


Figure 5: Per-role F1 over tensors.

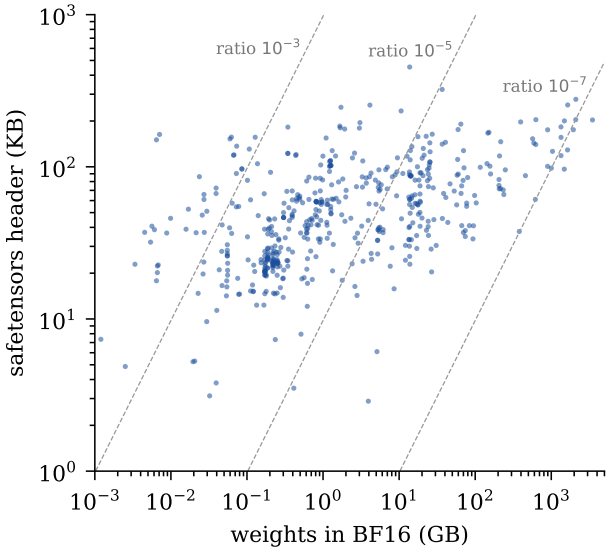


Figure 6: Header size against weight size for the 510 manifests. Dashed lines mark constant ratios. The median header is 46 KB and the largest 453 KB.

Name-pattern rules reach 69.3%. With names hashed, those rules have nothing to work with, whereas our method loses 1.8 points and 374 models still reach 99%. Most of the work is therefore done by structure and shapes rather than vocabulary.

Figure 5 breaks the result down by role. Attention, feed-forward, mixtures of experts and norms exceed 96 F1 with names and stay above 91 without them. Recurrent cells reach 100 with the lexicon but only 35 without it, because an LSTM whose input and hidden widths are equal has exactly the shapes of a feed-forward block that stores a second, integer copy of its weights, as I-BERT does. Cross-attention falls from 98 to 71 without names for a similar reason: when encoder and decoder share a width, the two attention regions of a decoder block have identical shapes, and only a name says which one reads the encoder. Output heads (54) and adapters (69) score lowest. Most of the remaining errors come from poolers and prediction transforms that the class names call heads but that look like a feed-forward or an embedding, and from single-matrix projectors inside the language model's folder.

Error analysis guided four changes made during this study, each checked on the whole corpus. Requiring the usual `ih`/`hh` naming before accepting a recurrent cell, when the lexicon is on, removed the I-BERT false positives and raised recurrent-cell F1 from 25 to 100. Treating three equal-width projections next to a convolution as attention fixed ConvBERT's span-based dynamic convolution, and excluding pointwise convolutions from the state-tensor test fixed conformer convolution modules that had been read as SSMs; together these raised convolution F1 from 77 to 83. Reading non-residual lists as heads or projectors raised head F1 from 39 to 54 and adapter F1 from 60 to 69. A rule that would have read every short convolution over the query, key and value channels as linear attention was tried and dropped: it fixed GLM-style gated delta layers but broke softmax attention layers that also use short convolutions.

The ten models below 50% each depart from the block conventions the rules assume rather than failing on a common architecture: ALBERT shares one layer across depth and stores no list at all, PoolFormer and PatchTSMixer have blocks without attention or feed-forward signatures, and xLSTM, XLNet, MobileBERT and MobileViTv2 use block layouts that no rule describes. The linear-attention layers of GLM-5-Next keep ordinary query, key and value projections and move the decay parameters into a sub-module, so the shape rules see attention; in EdgeTAM's perceiver the opposite happens. For some modules the ground truth itself

	d	layers	FFN	experts	vocab	heads	KV heads	head size
Manifest only, all models	92.7	92.1	90.5	97.3	87.2	72.4	69.9	66.4
number of models	467	441	336	73	359	445	226	152
Manifest only, single stack	94.7	96.3	93.5	96.4	85.6	70.4	63.9	57.1
Shapes only	92.7	92.1	90.5	94.5	84.7	72.4	69.9	66.4
With configuration fusion	–	–	–	–	–	95.7	98.2	96.7
Identifiable from shapes	yes	yes	yes	yes	yes	only with per-head parameters		

Table 3: Exact-match accuracy (%) of hyperparameters against the configuration. FFN is evaluated on dense models only.

is debatable. Kimi Delta Attention and the Bailing hybrid store the same tensors as the GLM layers but their classes end in `Attention`, so the class-name oracle calls them attention.

8.2 RQ2: Stacks and block types

98.6% of ground-truth block lists (1230 of 1248) are found at the right path, and every matched list has the right number of blocks. Precision is lower (91.4%) because the algorithm also reports repeated sub-blocks inside convolutional backbones, such as ResNet bottleneck sub-layers and SEANet residual units, which the ground-truth convention does not count. The block-type partition is exact for 92.5% of matched lists. Most of the misses are lists whose first block differs only in a module that the class signature sees and the tensor signature does not, such as a relative-position bias owned by layer 0.

8.3 RQ3: Quantized storage

Under NVFP4, FP8 and GPTQ storage the results equal those at full precision, and parameter totals stay exact for all 510 models (Table 2). This also holds with names hashed, because packed weights and their scales are recognized by dtype and shape ratios rather than by suffixes such as `weight_scale`. The point matters in practice. Hub pages count packed elements as parameters, so a 4-bit checkpoint is often shown at a fraction of its real size (Section 9).

8.4 RQ4: Identifiability, configuration fusion and consistency

Table 3 separates the fields shapes can identify from those they cannot. Width, depth, FFN size and expert count are correct for 90 to 97% of models, and more often when a model has a single stack, because then there is no doubt about which component the configuration describes. From shapes alone, the head count and head size are right about 70% of the time. The default head size is right more often (78%) than a norm vector of plausible length (69%). In latent attention, the norm before the low-rank key–value projection has the latent width rather than the head width, which is a concrete case of the limit described in Section 3.1.

With configuration fusion, head count, key–value head count and head size are right for 95.7%, 98.2% and 96.7% of models. Since the ground truth comes from the same configuration, these numbers measure the matching step, that is, whether the right sub-configuration is attached to the backbone. The backbone found a match in 455 of 510 models, and 872 of 1106 components found one. The more informative result is the consistency check. Of the 636 matched components with attention, 604 (95.0%) have configurations whose head fields multiply out to the projection widths in the weights. The 32 disagreements are concentrated in latent-attention models (DeepSeek-V2, V3 and V4, Kimi, MiniCPM3, Mistral 4, LongCat-Flash), where `head_dim` or `num_key_value_heads` refers to the query–key or latent geometry rather than to a projection width, in Falcon's fused multi-query layout, and in SAM-style windowed vision encoders. None

Model	Family	Params	Header	Recovered structure
Nemotron 3.5 Super VL	prefix LM	121.5 B	130 KB	ViT 32 L → projector → 40 SSM, 40 MoE, 8 attn.
Nemotron 3.5 Transcribe	prefix LM	32.4 B	154 KB	3 encoders → projector → 23 SSM, 23 MoE, 6 attn.
Nemotron 3.5 ASR streaming	transducer	638 M	57 KB	24 conformer blocks, LSTM, RNN-T joint
Parakeet TDT 0.6B v3	transducer	627 M	63 KB	same encoder, TDT joint, 5 duration bins
Nemotron 3 Embed 1B NVFP4	encoder	1.14 B	41 KB	16 blocks, 112 NVFP4 linear layers
FastH3 Omni FP8†	diffusion	33.1 B	84 KB	2 refiner + 50 DiT blocks, 300 FP8 tensors
Kumo Tabular (large)†	encoder	214 M	41 KB	row embedding → 24-layer backbone
Kumo Relational†	encoder	29 M	24 KB	rows → message passing → 12 layers

Table 4: Anatomies recovered for models released in 2026. Manifests marked † were rebuilt from public source code and configuration because `transformers` has no reference implementation for them.

of these configurations is wrong; each contains a field whose meaning is family-specific. A tool that trusted the configuration blindly would report wrong KV-cache sizes for exactly these models, which is why the explorer shows the disagreement instead of resolving it.

8.5 RQ5: Cost

The median safetensors header is 46 KB, the 95th percentile 164 KB and the largest 453 KB, for models of up to 3.5 TB in BF16 (Figure 6). Recovery takes 11.4 ms for the median model on a single JavaScript thread, 60 ms at the 95th percentile and 198 ms at most. In the explorer, network round trips dominate: a single-file model needs four requests (listing, configuration and two range requests), and a sharded model two range requests per shard, issued in parallel.

9 Case studies

Table 4 lists eight models released in 2026 that we studied in detail. They were not in the corpus, but they were not held out from development either: we studied them while extending the rules, and several parts of Section 5, including transducer joints, timestep embedders, message-passing layers and adapter components, were written because of them. Every change was checked on the full corpus for regressions. The case studies therefore show that the representation is rich enough for these architectures, while the unbiased accuracy estimate is the one in Section 8.

The Super VL backbone mixes three block types in one list, and the recovered type sequence reproduces the configuration's `layers_block_type` exactly. The projector's input width, 5120, is four times the vision width, and the algorithm reports a 2×2 merge, matching the configuration's `downsample_ratio` of 0.5. Experts hold 112.7 B of the 121.5 B parameters. With the configured top-22 routing this gives 13.6 B active parameters per token, including the vision tower and both embedding tables, against the 12 B in the model's name. Only 8 of the 88 layers keep a KV cache, each with two key–value heads, so the cache costs 8 KB per token in BF16, an eleventh of what it would be if every layer used the same attention.

For both speech transducers the joint network is found from three shapes, and the number of duration bins falls out as a difference. Parakeet's joint output has 8198 rows and its label table 8193, giving the five durations $\{0, \ldots, 4\}$ of its configuration, while the streaming model's two sizes are equal, as in a standard RNN-T. Parakeet's parameter total differs from the Hub's figure by 24, the number of batch-norm step counters, one per conformer block. The NVFP4 embedding model has 1.14 B logical parameters, matching its model card, while the Hub reports 0.7 B because it counts packed `U8` elements. That it is an encoder cannot be seen in the weights; its configuration's `is_causal: false` settles it, and the explorer shows the family as read from the configuration.

Besides the new components, four fixes to existing rules came from these models: the residual width of a block is taken from its norms, `nn.Sequential` children are no longer split as sub-layers, a module that wraps two different sub-blocks is split, and in repositories with several top-level folders a component is never raised above its folder. One part stays unexplained: the streaming model's language-prompt projector remains unattached, because its prompt table lives outside the components the manifest reveals.

10 Discussion and limitations

Class names are an independent source of labels but not a perfect one. Some implementations place a position table inside a class called `...Embeddings` and others inside the encoder; some call a pooler a `...Head`; and linear-attention modules are named `...Attention` in some families and `...GatedDeltaNet` in others. We merged roles where the library is inconsistent and kept the fallback list short, but per-role numbers for heads and adapters should be read with this in mind. The headline numbers are robust to it, because these roles carry little of the parameter mass.

All corpus manifests come from `transformers` implementations, which share naming habits. The shapes-only condition is the main protection against overfitting to those habits, and its 1.8-point drop suggests that the structural rules transfer. Default configurations are not trained checkpoints, but their shapes, which are all the method reads, follow the same rules. Fusion accuracy is measured against the same configuration that supplies the fields, so it validates the matching step and not the configuration; the consistency check is the independent part, and a richer check could model latent attention explicitly.

GGUF and ONNX files are not handled yet, although both carry comparable metadata, and old non-zip PyTorch pickles are rejected. Order inside a block and activation functions remain unknown, and compute estimates assume standard computation per role. The method reads only what a hub already serves publicly. It never downloads or executes weights, and the PyTorch reader interprets a fixed subset of pickle opcodes as data without resolving Python callables.

11 Conclusion

A tensor manifest is small, safe and almost always available, and it carries more of an architecture than one might expect. With the module tree, a few dozen shape rules and width matching, we recover blocks, block types, regions, data paths and parameter counts for 510 architectures at 96.8% role accuracy, keep most of that accuracy without any names, and stay exact on parameter counts under current quantized formats. The same analysis makes the limits explicit: head geometry, masking and routing are not in the shapes. Taking them from the configuration closes most of the gap, and checking the configuration against the weights exposes the families whose fields mean something else. We hope the benchmark and the explorer make it routine to look inside a model before downloading it.

Availability. The explorer runs at https://joessattes.github.io/ai-model-explorer/ and the checkpoint benchmark at https://joessattes.github.io/ai-model-explorer/bench.html. Source code, the corpus builder, the evaluation scripts and the sources of this paper are at https://github.com/JoesSattes/ai-model-explorer under the MIT license. The corpus results are reproduced by `python research/build_corpus.py corpus.jsonl` followed by `node research/eval.js corpus.jsonl` with the flags `-obf -nolex`, `-quant nvfp4`, `-v1` and `-cfg`; the checkpoint results by `node research/realbench.js export.jsonl corpus.jsonl`.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[2] Aeree Cho, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng Chau. Transformer explainer: Interactive learning of text-generative models. *arXiv preprint arXiv:2408.04619*, 2024.

[3] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, 2024.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[7] Google AI Edge. Model explorer: Intuitive and hierarchical visualization of model graphs. https://ai.google.dev/edge/model-explorer, 2024.

[8] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[10] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Interspeech*, 2020.

[11] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[12] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, and Yuan Xie. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 385–399, 2020.

[13] Hugging Face. safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors, 2022.

[14] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.

[15] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[16] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

[17] NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

[18] NVIDIA. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[19] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[20] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 50817–50847, 2025.

[21] Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. Fast conformer with linearly scalable attention for efficient speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023.

[22] Lutz Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. https://github.com/lutzroeder/netron, 2017.

[23] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[24] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[25] Sattaya Singkul. AI Model Explorer. https://joessattes.github.io/ai-model-explorer/, 2026.

[26] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[27] Hainan Xu, Fei Jia, Somshubra Majumdar, He Huang, Shinji Watanabe, and Boris Ginsburg. Efficient sequence transduction by jointly predicting tokens and durations. In *International Conference on Machine Learning*, 2023.

[28] Mengjia Yan, Christopher W. Fletcher, and Josep Torrellas. Cache telepathy: Leveraging shared resource attacks to learn DNN architectures. In *29th USENIX Security Symposium*, pages 2003–2020, 2020.

[29] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

[30] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. HuRef: HUman-REadable fingerprint for large language models. In *Advances in Neural Information Processing Systems*, 2024.

[31] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

A Corpus construction

The model class for each configuration type is chosen in this order: image-text-to-text, multimodal LM, speech sequence-to-sequence, sequence-to-sequence LM, TDT, RNN-T, causal LM, CTC, masked LM, image classification, text-to-waveform, object detection, base model. Instantiation uses `_from_config` under `torch.device("meta")` with a 90-second limit. Of the 240 types that do not instantiate, 177 are sub-configurations without a model class of their own, such as vision or audio towers registered separately, and the rest fail on required sub-configurations or optional dependencies. Manifests are deduplicated by the exact list of names and shapes.

B Synthetic quantized storage

The rewrite applies to every 2-D tensor named `*.weight` whose input dimension is a multiple of 32 and output dimension a multiple of 8, excluding embeddings, heads and norms. NVFP4 replaces $[o, i]$ by `U8` $[o, i/2]$, `F8_E4M3` $[o, i/16]$ and two `F32` scalars. FP8 uses `F8_E4M3` $[o, i]$ and `F32` $[o, 1]$. GPTQ uses `qweight` as `I32` $[i/8, o]$, `qzeros` $[i/128, o/8]$, `scales` as `F16` $[i/128, o]$ and `g_idx` $[i]$. Under name hashing these suffixes are hashed too, so recognition relies on dtypes and shape ratios.

C Additional interface view

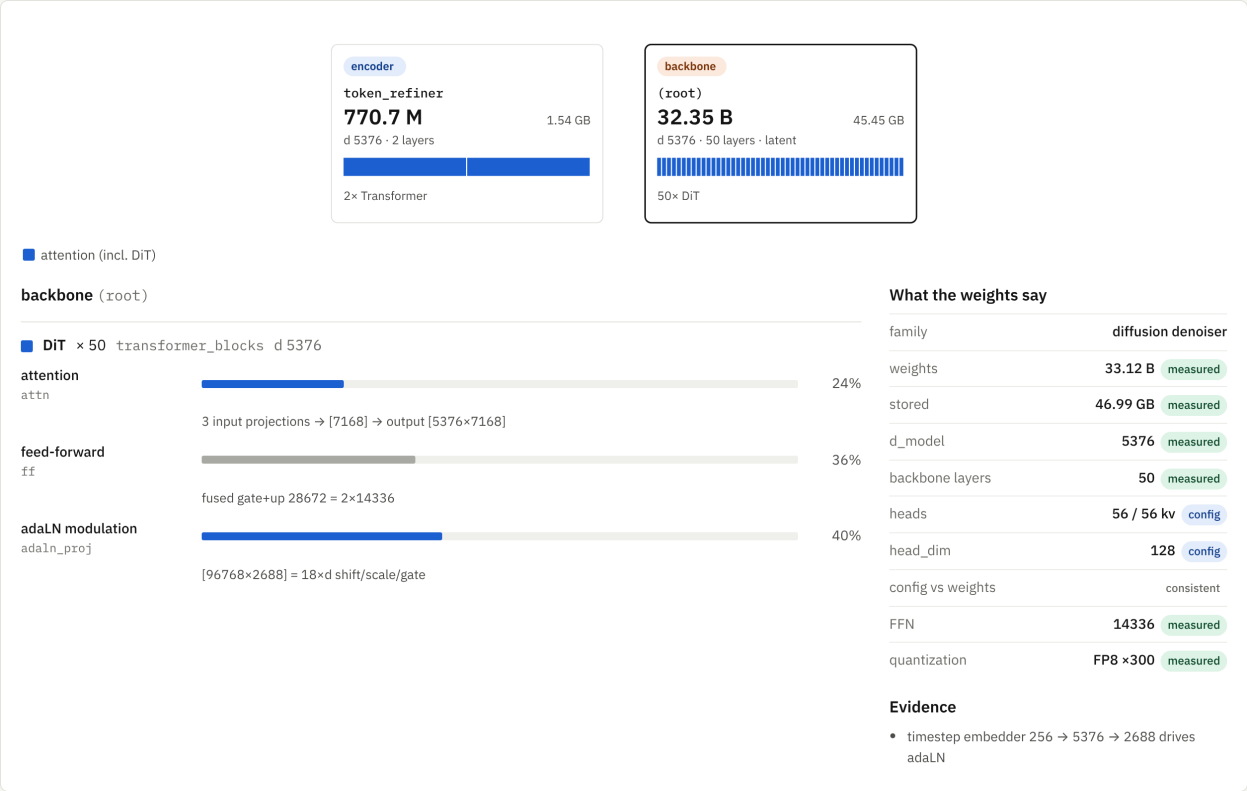


Figure 7: Anatomy view of the FastH3 Omni FP8 denoiser (reconstructed manifest): a two-block token refiner and a 50-block DiT backbone. The head geometry is measured from per-head norms, and the FP8 storage of 300 tensors is shown with logical and stored sizes.